

AMBA – APB PROTOCOL

APB MASTER

```
//=====
// APB MASTER MODULE - WRITE + READ CYCLE (3-Slave Version)
//=====

module apb3_master (
    input wire    PCLK,
    input wire    PRESETn,
    output reg     PSEL,
    output reg     PENABLE,
    output reg     PWRITE,
    output reg [7:0] PADDR,
    output reg [7:0] PWDATA,
    input wire [7:0] PRDATA,
    input wire     PREADY
);

// FSM States

reg [2:0] state;

localparam IDLE    = 3'b000;
localparam SETUP_WR = 3'b001;
localparam ENABLE_WR = 3'b010;
localparam SETUP_RD = 3'b011;
localparam ENABLE_RD = 3'b100;
localparam DONE    = 3'b101;

// Track which slave (cycle counter)
```

```
reg[1:0] cycle; // 0 → Slave1, 1 → Slave2, 2 → Slave3
```

always @(posedge PCLK or negedge PRESETn) begin

```
if (!PRESETn) begin
```

```
state <= IDLE;
```

```
cycle  <= 0;
```

```
PSEL <= 0;
```

```
PENABLE <= 0;
```

```
PWRITE <= 0;
```

```
PADDR <= 8'd0;
```

```
PWDATA <= 8'd0;
```

end else begin

case (state)

//-----

```
// IDLE → Prepare new write for each slave
```

//-----

IDLE: begin

```
PSEL <= 1;
```

```
PWRITE <= 1; // Start write
```

```
// Target each slave based on current cycle
```

case (cycle)

2'd0: begin

```
PADDR <= 8'h05; // Slave 1 address
```

```
PWDATA <= 8'h11;
```

end

2'd1: begin

```
PADDR <= 8'h15; // Slave 2 address
```

```

        PWDATA <= 8'h22;

    end

    2'd2: begin

        PADDR <= 8'h25; // Slave 3 address

        PWDATA <= 8'h33;

    end

    default: begin

        PADDR <= 8'h00;

        PWDATA <= 8'h00;

    end

endcase

state <= SETUP_WR;

end

//-----

// SETUP WRITE

//-----

SETUP_WR: begin

    PENABLE <= 1;

    state <= ENABLE_WR;

end

//-----

// ENABLE WRITE

//-----

ENABLE_WR: begin

    if (PREADY) begin

        $display("WRITE -> ADDR: %h DATA: %h", PADDR, PWDATA);
    end
end

```

```

        PENABLE <= 0;

        PSEL  <= 0;

        PWRITE <= 0;

        state <= SETUP_RD;

    end

end

//-----

// SETUP READ

//-----

SETUP_RD: begin

    PSEL  <= 1;

    PWRITE <= 0;

    PENABLE <= 1;

    state <= ENABLE_RD;

end

//-----

// ENABLE READ

//-----

ENABLE_RD: begin

    if (PREADY) begin

        $display("READ <- ADDR: %h DATA: %h", PADDR, PRDATA);

        PENABLE <= 0;

        PSEL  <= 0;

        state <= DONE;

    end

end

end

```

```

//-----
// DONE → Move to next slave or finish all
//-----

DONE: begin

    PENABLE <= 0;

    PSEL  <= 0;

    PWRITE <= 0;


    if (cycle < 2) begin

        cycle <= cycle + 1;

        state <= IDLE; // Start next transaction

    end else begin

        $display("All 3 slave transactions complete.");

        state <= DONE; // Hold final state

    end

end

endcase

end

end

endmodule

```

APB SLAVE 3

```

//=====
// APB SLAVE MODULE
//=====

module apb3_slave (
    input wire    PCLK,
    input wire    PRESETn,

```

```
input wire    PSEL,
input wire    PENABLE,
input wire    PWRITE,
input wire [7:0] PADDR,
input wire [7:0] PWDATA,
output reg [7:0] PRDATA,
output reg    PREADY
);
```

```
// Simple 256x8 memory
reg [7:0] memory [0:255];
```

```
always @(posedge PCLK or negedge PRESETn) begin
    if (!PRESETn) begin
        PRDATA <= 8'd0;
        PREADY <= 1'b0;
    end else begin
        if (PSEL && PENABLE) begin
            PREADY <= 1'b1;
            if (PWRITE) begin
                memory[PADDR] <= PWDATA; // WRITE operation
            end else begin
                PRDATA <= memory[PADDR]; // READ operation
            end
        end else begin
            PREADY <= 1'b0;
        end
    end
end
```

```
endmodule
```

APB DECODER

```
//=====
// APB Address Decoder for 3 Slaves
//=====

module apb3_decoder (
    input  wire [7:0] PADDR,
    output reg      PSEL1,
    output reg      PSEL2,
    output reg      PSEL3
);

    always @(*) begin
        // Default all selects off
        PSEL1 = 0;
        PSEL2 = 0;
        PSEL3 = 0;

        case (PADDR[7:4])
            4'h0: PSEL1 = 1; // Slave 1 handles 0x00-0x0F
            4'h1: PSEL2 = 1; // Slave 2 handles 0x10-0x1F
            4'h2: PSEL3 = 1; // Slave 3 handles 0x20-0x2F
            default: begin
                PSEL1 = 0;
                PSEL2 = 0;
                PSEL3 = 0;
            end
        endcase
    end
endmodule
```

```
end
```

```
endmodule
```

APB TOP MODULE

```
//=====
```

```
// APB FPGA TOP MODULE - Single Master, 3 Slaves
```

```
//=====
```

```
module apb_fpga_top (
```

```
    input wire    PCLK,
```

```
    input wire    PRESETn,
```

```
    output wire [15:0] debug_out
```

```
);
```

```
// -----
```

```
// Internal APB signals
```

```
// -----
```

```
wire [7:0] PADDR, PWDATA, PRDATA;
```

```
wire    PWRITE, PENABLE;
```

```
wire [2:0] PSEL;
```

```
wire    PREADY;
```

```
// Outputs from each slave
```

```
wire [7:0] PRDATA1, PRDATA2, PRDATA3;
```

```
wire    PREADY1, PREADY2, PREADY3;
```

```
// -----
```

```
// Instantiate APB Master
```

```
// -----
```

```
apb3_master master_inst (
```



```

        .PCLK(PCLK),
        .PRESETn(PRESETn),
        .PADDR(PADDR),
        .PWDATA(PWDATA),
        .PRDATA(PRDATA),
        .PWRITE(PWRITE),
        .PENABLE(PENABLE),
        .PSEL(PSEL[0]),
        .PREADY(PREADY)
    );

// -----
// Address Decoder
// -----

apb3_decoder decoder_inst (
    .PADDR(PADDR),
    .PSEL1(PSEL[0]),
    .PSEL2(PSEL[1]),
    .PSEL3(PSEL[2])
);

// -----
// 3 APB Slaves
// -----

apb3_slave slave1 (
    .PCLK(PCLK),
    .PRESETn(PRESETn),
    .PSEL(PSEL[0]),
    .PENABLE(PENABLE),
    .PWRITE(PWRITE),

```

```
.PADDR(PADDR),  
.PWDATA(PWDATA),  
.PRDATA(PRDATA1),  
.PREADY(PREADY1)  
);
```

```
apb3_slave slave2 (  
    .PCLK(PCLK),  
    .PRESETn(PRESETn),  
    .PSEL(PSEL[1]),  
    .PENABLE(PENABLE),  
    .PWRITE(PWRITE),  
    .PADDR(PADDR),  
    .PWDATA(PWDATA),  
    .PRDATA(PRDATA2),  
    .PREADY(PREADY2)  
);
```

```
apb3_slave slave3 (  
    .PCLK(PCLK),  
    .PRESETn(PRESETn),  
    .PSEL(PSEL[2]),  
    .PENABLE(PENABLE),  
    .PWRITE(PWRITE),  
    .PADDR(PADDR),  
    .PWDATA(PWDATA),  
    .PRDATA(PRDATA3),  
    .PREADY(PREADY3)  
);
```

```

// -----
// Data and Ready multiplexing
// -----
assign PRDATA = (PSEL[0]) ? PRDATA1 :
    (PSEL[1]) ? PRDATA2 :
    (PSEL[2]) ? PRDATA3 : 8'h00;

assign PREADY = (PSEL[0]) ? PREADY1 :
    (PSEL[1]) ? PREADY2 :
    (PSEL[2]) ? PREADY3 : 1'b0;

// -----
// Debug signal to preserve design hierarchy
// -----
reg [15:0] debug_reg;

always @(posedge PCLK or negedge PRESETn) begin
    if (!PRESETn)
        debug_reg <= 16'h0000;
    else
        // Mix key internal signals to force logic retention
        debug_reg <= {PSEL, PWRITE, PENABLE, PREADY, PRDATA[3:0], PADDR[3:0]};
    end

assign debug_out = debug_reg;

endmodule

```

APB TESTBENCH

```
// TESTBENCH - SINGLE MASTER, THREE SLAVES APB SYSTEM
```

```

//=====

`timescale 1ns/1ps

module apb3_tb;

    // -----
    // APB Signal Declarations
    // -----

    reg    PCLK;
    reg    PRESETn;
    wire [7:0] PADDR;
    wire [7:0] PWDATA;
    wire [7:0] PRDATA;
    wire    PWRITE;
    wire    PENABLE;
    wire    PSEL;
    wire    PREADY;

    // Decoder output select lines
    wire    PSEL1, PSEL2, PSEL3;

    // Each slave's data + ready lines
    wire [7:0] PRDATA1, PRDATA2, PRDATA3;
    wire    PREADY1, PREADY2, PREADY3;

    // -----
    // Clock Generation
    // -----

    initial begin
        PCLK = 0;

```

```
    forever #5 PCLK = ~PCLK; // 100MHz clock (10ns period)
end
```

```
// -----
```

```
// Reset Logic
```

```
// -----
```

```
initial begin
```

```
    PRESETn = 0;
```

```
    #20 PRESETn = 1; // release reset after 20ns
```

```
end
```

```
// -----
```

```
// Instantiate APB Master
```

```
// -----
```

```
apb3_master master_inst (
```

```
    .PCLK(PCLK),
```

```
    .PRESETn(PRESETn),
```

```
    .PADDR(PADDR),
```

```
    .PWDATA(PWDATA),
```

```
    .PRDATA(PRDATA),
```

```
    .PWRITE(PWRITE),
```

```
    .PENABLE(PENABLE),
```

```
    .PSEL(PSEL),
```

```
    .PREADY(PREADY)
```

```
);
```

```
// -----
```

```
// Instantiate Address Decoder
```

```
// -----
```

```
apb3_decoder decoder_inst (
```

```
.PADDR(PADDR),  
.PSEL1(PSEL1),  
.PSEL2(PSEL2),  
.PSEL3(PSEL3)  
);
```

```
// -----  
// Instantiate 3 APB Slaves  
// -----
```

```
apb3_slave slave1_inst (  
    .PCLK(PCLK),  
    .PRESETn(PRESETn),  
    .PSEL(PSEL1),  
    .PENABLE(PENABLE),  
    .PWRITE(PWRITE),  
    .PADDR(PADDR),  
    .PWDATA(PWDATA),  
    .PRDATA(PRDATA1),  
    .PREADY(PREADY1)  
);
```

```
apb3_slave slave2_inst (  
    .PCLK(PCLK),  
    .PRESETn(PRESETn),  
    .PSEL(PSEL2),  
    .PENABLE(PENABLE),  
    .PWRITE(PWRITE),  
    .PADDR(PADDR),  
    .PWDATA(PWDATA),  
    .PRDATA(PRDATA2),
```

```

        .PREADY(PREADY2)
    );

    apb3_slave slave3_inst (
        .PCLK(PCLK),
        .PRESETn(PRESETn),
        .PSEL(PSEL3),
        .PENABLE(PENABLE),
        .PWRITE(PWRITE),
        .PADDR(PADDR),
        .PWDATA(PWDATA),
        .PRDATA(PRDATA3),
        .PREADY(PREADY3)
    );

    // -----
    // Combine slave outputs to master bus
    // -----
    assign PRDATA = PSEL1 ? PRDATA1 :
        PSEL2 ? PRDATA2 :
        PSEL3 ? PRDATA3 : 8'h00;

    assign PREADY = PSEL1 ? PREADY1 :
        PSEL2 ? PREADY2 :
        PSEL3 ? PREADY3 : 1'b0;

    // -----
    // Simulation Control
    // -----
    initial begin

```

```
$dumpfile("apb_wave.vcd");
```

```
$dumpvars(0, apb3_tb);
```

```
#3000; // allow enough time for all 3 slaves
```

```
$finish;
```

```
end
```

```
endmodule
```